Dt:02.01.2026

Worked: Arsildo Veliu

Smart Library Testing Documentation

1. Code Coverage Testing

Method Tested: `canBorrowItemType()`

This method checks whether a member can borrow an item based on valid member ID and item details, member active status, borrow capacity, overdue items, and membership-based value limits (Student: $100, Staff: $500, Guest: $50).

Statement Coverage

Statement coverage is achieved using a valid borrowing scenario where all conditions are satisfied. This test ensures every executable statement in the method is run at least once.

Branch Coverage

Branch coverage tests all decision points in both outcomes. The following validation cases are covered: invalid member ID (memberId <= 0), null item type, negative item value, and member not found in storage. Each case causes the method to return false, ensuring all validation branches are executed.

Condition Coverage

The final decision in the method is based on four conditions:

isActive && hasCapacity && noOverdue && valueLimitOk

Each condition is tested when it evaluates to false: inactive member, borrow limit reached, member has overdue items, and item value exceeds membership limit. A base test case confirms the method returns true when all conditions are true.

MC/DC (Modified Condition/Decision Coverage)

MC/DC is achieved by changing one condition at a time while keeping all others true:

- Base case: all conditions true → result true
- isActive false → result false
- hasCapacity false → result false
- noOverdue false → result false
- valueLimitOk false → result false

This demonstrates that each condition independently affects the result.

Additional Tests

Extra tests verify that item value limits work correctly for different membership types: staff members can borrow higher-value items, while guest members are restricted to lower-value items.

2. Equivalence Class Testing

Method Tested: `getMembershipEligibility()`

This method returns a membership status string based on number of current borrows, overdue status, and active status.

Identified Equivalence Classes

- **Invalid borrow count** (currentBorrows < 0)
- **Inactive membership** (regardless of other parameters)
- **Active membership with overdue items**
- **Excellent standing** (0 borrows, active, no overdue)
- **Good standing** (1–2 borrows, active, no overdue)
- **Moderate usage** (3–4 borrows, active, no overdue)
- **Heavy usage** (5+ borrows, active, no overdue)

Representative and boundary values from each class are tested to ensure correct behaviour. For example, good standing tests both 1 borrow (minimum) and 2 borrows (maximum), while heavy usage tests 5 borrows (boundary), 10 (typical), and 100 (extreme).

3. Boundary Value Testing

Method Tested: `getDaysUntilDue()`

This method calculates how many days remain until an item is due.

Boundary Tested

The key boundary occurs when the due date equals the current date. Three boundary cases are tested:

- Due yesterday → -1
- Due today → 0
- Due tomorrow → 1

Additional Boundary Cases

Tests for null item code, empty item code, no active borrow record, and returned item ensure the method handles invalid and edge cases correctly.

Summary

The testing approach combines coverage, equivalence class, and boundary value testing to validate the circulation service. Code coverage ensures all logic paths are executed, equivalence class testing efficiently covers input categories, and boundary value testing focuses on error-prone edge cases. Together, these techniques provide comprehensive test coverage for the Smart Library system.